

TCC · DOCUMENTO DE BANCO DE DADOS

ControlaAI TCC — Banco de Dados PostgreSQL (Colunas, Tabelas, Relações e Chaves)

Documento estratégico em 8 páginas e 3 capítulos: modelagem PostgreSQL, infraestrutura VPS Railway, autenticação, tabelas, colunas, relações, chaves e dados reais usados na apresentação.

SISTEMA

Controla.AI — Assistente Financeiro Pessoal com IA

ESTRUTURA

Cap. 1 Introdução · Cap. 2 VPS/Railway/Postgres/Redis · Cap. 3 Tabelas, PK/FK, fluxos e seeds

EQUIPE

Davi Almeida
Leonardo Sena
Gustavo Biscoto

Agosto / 2026
Curitiba — PR

1 **Conhecendo o Controla.AI**

Problema · Solução · Papel do banco · Pilares

Controlar gastos sem formulários: o usuário manda mensagem no WhatsApp; a IA interpreta, grava no PostgreSQL e confirma. Este capítulo explica por que o banco é o centro do sistema.

16

Tabelas

18

Relações FK

13

Enums PG

3

Capítulos

Problema

Apps de finanças exigem muitos cliques; a maioria abandona o controle do dinheiro.

Solução

Texto/áudio/foto no WhatsApp + painel web + importação de PDF de extrato.

Banco

PostgreSQL 15+ na Railway (VPS), administrado via DBeaver com SSL.

Pilares da modelagem

- ✓ **users** é o hub: quase todas as FKs apontam para `users.id` (UUID).
- ✓ **Soft delete de categoria** com `ON DELETE SET NULL` preserva histórico de lançamentos.
- ✓ **Enums nativos** validam plano, tipo de transação, status Stripe etc. no servidor.
- ✓ **Auditoria:** `ai_logs`, `whatsapp_messages` e `user_consent`s registram cada ação sensível.

O que o professor encontra neste documento

Tabelas e colunas com tipos; PK/FK e cardinalidade; dados seed; VPS Railway + Redis; autenticação (bcrypt + JWT + LGPD); fluxos WhatsApp → parser → transactions.

2

Infraestrutura e Hospedagem

PostgreSQL · Redis · DBever · Deploy

Produção na **Railway** (VPS gerenciada): Postgres + Redis com backup e SSL. A equipe administra o schema pelo **DBever** local via URL pública (sslmode=require). Frontend na **Vercel**; backend Node/Fastify na Railway.

PostgreSQL Railway

PG 15+ · extensão pgcrypto · pool ~10 · Drizzle ORM · migrations em drizzle/ · host interno *.railway.internal:5432 · proxy público para DBever.

Redis Railway

Cache de sessões/KPIs · auth default+senha · latência baixa na rede interna · base para filas futuras do parser em lote.

DBever — console da equipe

Queries ad-hoc, seeds, export CSV, inspeção de ai_logs e whatsapp_messages. Script completo: backend/scripts/novo-banco-railway-COMPLETO.sql.

Stack resumida

Camada	Tecnologia	Papel
Frontend	React · Vite · Tailwind	Dashboard, login, metas, chat
Backend	Node · Fastify · TS	API REST, JWT, webhooks
Banco	PostgreSQL · Drizzle	16 tabelas relacionais
Cache	Redis	Sessão / KPIs
IA	GPT-4o-mini · Whisper	Parser, chat, áudio
WhatsApp	Baileys	Canal oficial
Pagamento	Stripe	Planos Pro/Premium
Deploy	Railway + Vercel	CI/CD produção

3

Autenticação ligada ao banco

Início do Capítulo 3 — segurança e LGPD

Senhas bcrypt (10 rounds)
Gravadas só em `users.password_hash`. Login usa `bcrypt.compare` — sem descriptografia.

JWT HS256 · 7 dias
Payload: `{userId, email}`. Middleware injeta usuário; toda query filtra por `user_id`.

Cadastro em transação ACID + LGPD

- 1

Aceite legal
Termos, Privacidade e LGPD (versão datada) via GET `/auth/legal`.
- 2

INSERT users + user_settings
Conta + preferências 1:1; UNIQUE(email) bloqueia duplicata.
- 3

3× user_consent
Versão, IP, user-agent, horário — prova de aceite LGPD.
- 4

COMMIT + JWT
Tudo ou nada; token devolvido ao frontend (Bearer).

Tabela	Colunas-chave	Uso no login/cadastro
users	email, password_hash, phone, plan	Busca + bcrypt + plano Stripe
user_settings	onboarding_completed, alertas	Onboarding e preferências
user_consent	consent_type, document_version	Auditoria LGPD
subscriptions	status, period_end	Plano pago ativo

Regra de ouro
Nenhuma consulta retorna dados sem `WHERE user_id = $jwt`. Role do app sem SUPERUSER.

Mapa das 16 tabelas (o que cada uma faz)

users

Titular. plan (free/pro/premium), phone→WhatsApp, stripe_customer_id.

user_settings (1:1)

Alertas 80/100%, tema, onboarding, saldo e tipo de renda.

categories

10 despesas + 4 receitas seed (user_id NULL) + personalizadas.

transactions

Coração financeiro: valor, tipo, origem, raw_message, parcelas.

goals + goal_checkpoints

Metas de limite/poupança + snapshot mensal de % e alertas.

budgets

UNIQUE(user_id, month): renda e teto de despesa do mês.

recurring_transactions

Contas fixas; job materializa em transactions.

whatsapp_messages

Inbound/outbound, mídia, FK opcional para transaction.

ai_logs

Modelo, tokens, cost_usd, duração — rastreo OpenAI.

ai_conversations

Histórico do chat web em JSONB.

Auxiliares

user_consents · financial_memory · document_imports · subscriptions · whatsapp_connection · whatsapp_sessions.

Dados dentro

Seeds: 8 usuários, +210 lançamentos (Leonardo), 5 metas, mensagens e logs IA.

PK universal: UUID `gen_random_uuid()`. Hub: **users**. CASCADE apaga filho com o dono; SET NULL preserva histórico (categorias, logs, msgs).

#	Origem	FK	→ Destino	Card.	ON DELETE
1	user_settings	user_id (PK)	users.id	1:1	CASCADE
2	transactions	user_id	users.id	1:N	CASCADE
3	transactions	category_id	categories.id	N:1	SET NULL
4	categories	user_id	users.id	1:N	CASCADE
5	goals	user_id	users.id	1:N	CASCADE
6	goals	category_id	categories.id	N:1	SET NULL
7	goal_checkpoints	goal_id	goals.id	1:N	CASCADE
8	budgets	user_id	users.id	1:N	CASCADE
9	recurring_tx	user_id / cat	users / cats	1:N	CASCADE
10	ai_conversations	user_id	users.id	1:N	CASCADE
11	whatsapp_messages	user_id	users.id	1:N	SET NULL
12	whatsapp_messages	transaction_id	transactions.id	0:1	SET NULL
13	ai_logs	user_id	users.id	1:N	SET NULL
14–18	+5 tabelas	memory, imports, subscriptions, consents, wa_sessions			CASCADE

Colunas essenciais de users e transactions

users	Tipo	Nota	transactions	Tipo	Nota
id	UUID	PK	id	UUID	PK
email	TEXT	UNIQUE	user_id	UUID	FK CASCADE
password_hash	TEXT	bcrypt	category_id	UUID	FK SET NULL
phone	TEXT	UNIQUE NULL	amount / type	NUM/ENUM	expense income
plan	ENUM	free/pro/premium	source	ENUM	whatsapp web ...

► **WhatsApp → lançamento**

- 1 Baileys upsert**
INSERT whatsapp_messages (inbound).
- 2 Resolver telefone**
users.phone (+55 / 9º dígito).
- 3 Parser OpenAI**
Valor/tipo/categoria → ai_logs.
- 4 INSERT transactions**
Liga transaction_id na mensagem.

► **Metas e alertas**

- 1 Cria goal**
Limite ou poupança + deadline.
- 2 A cada gasto**
Soma mês e %; checkpoint opcional.
- 3 Alerta 80%**
user_settings + flag no checkpoint.
- 4 Poupança**
Janela [created_at, deadline].

► **Chat IA, Stripe e PDF**

Chat web: mesmo agente financeiro; histórico em ai_conversations (JSONB); insights agregam transactions × budgets × recurring.

Stripe: webhook HMAC → UPSERT subscriptions → UPDATE users.plan. Cancelamento volta plan=free.

PDF extrato: document_imports (pending→completed) · pdf-parse · GPT · N transactions.

Para a banca

Mostre no DBeaver: SELECT em transactions do Leonardo + JOIN categories + goals — prova relação, dados e isolamento por user_id.

8 Usuários	+210 Tx Leonardo	5 Metas	6 Orçamentos
---------------	---------------------	------------	-----------------

Usuário	E-mail	Senha	Plano
Admin	admin@admin.com	123456	premium
Davi	davi.almeida@unicesumar.edu.br	123456	premium
Leonardo	leonardo.sena@unicesumar.edu.br	123456	premium
Gustavo	gustavo.biscoto@unicesumar.edu.br	123456	premium
Demos	marina / carlos / juliana / roberto @email.com	123456	free/pro

- ✓ Leonardo: 6 meses (mar-ago/2026), 5 metas, checkpoints, amostras WhatsApp e ai_logs.
- ✓ 14 categorias padrão + settings por usuário + conexão WhatsApp singleton.
- ✓ Reproduzir: executar novo-banco-railway-COMPLETO.sql no DBeaver (Railway SSL).

Fechamento dos 3 capítulos

Cap.1 problema/solução e pilares · **Cap.2** VPS Railway, Postgres, Redis, DBeaver · **Cap.3** auth, 16 tabelas, colunas, 18 FKs, fluxos e seeds — tudo que a banca pede sobre tabelas, colunas, dados, relações e chaves, em 8 páginas.